Reentrancy Redux: The Evolution of Real-World Reentrancy Attacks on Blockchains

Yuqi Liu
University of British Columbia
yuqil@ece.ubc.ca

Rui Xi
University of British Columbia
xirui801@ece.ubc.ca

Karthik Pattabiraman
University of British Columbia
karthikp@ece.ubc.ca

***Abstract*—Reentrancy attacks remain a persistent threat to blockchain smart contracts today, causing significant financial losses despite numerous defense mechanisms. This paper presents a comprehensive analysis of 73 real-world reentrancy attacks on EVM-compatible blockchains from 2016 to 2024, investigating the factors contributing to their continued prevalence. Through integrated qualitative and quantitative analyses, we identify key trends in exploited vulnerabilities, track the evolution of attacker techniques, and expose a widening gap between academic research and real-world practice. Our findings reveal that reentrancy attacks are more diverse and sophisticated than previously understood, frequently involving complex interactions across multiple contracts, projects, and even blockchains. Critically, we highlight how attackers are adapting to bypass traditional detection and defense techniques. This research provides crucial insights into the evolving threat landscape, challenges outdated assumptions, and offers practical guidelines for developing more robust and effective reentrancy defenses.**

***Index Terms*—Reentrancy, Smart Contract Vulnerability, Ethereum, Real-World Attacks, Blockchain Security**

I. Introduction

Blockchain technology is a decentralized, transparent ledger system designed for secure and immutable record-keeping. Among various networks, Ethereum [1] introduced *smart contracts*—self-executing programs deployed on-chain to automate transactions and agreements. However, as smart contracts have become prevalent, their security vulnerabilities have also grown [2], [3], leading to billions of dollars in financial losses.

Reentrancy attacks are among the most prevalent and well-documented smart contract vulnerabilities [2]. They occur when a malicious contract X calls another contract Y, which then invokes a callback into X before updating its own state. Exploiting this inconsistency, X can manipulate interactions to deceive Y into unintended behaviors, often resulting in fund loss. At least 73 reentrancy attacks are recorded on Ethereum Virtual Machine (EVM) compatible blockchains [4]–[6].

While there are many tools for detecting reentrancy attacks, they face intrinsic limitations. For instance, Oyente [7] applies overly general patterns, leading to high false positives [8]; Slither [9] restricts its focus to Ether transfers, missing vulnerabilities in ERC-20 token transactions; and dynamic detectors like Sereum [10] require modifications to the underlying infrastructure, limiting their practical deployment. As a result, reentrancy attacks remain an active and serious threat.

This study focuses specifically on reentrancy attacks for three key reasons. First, they are among the earliest and most widely recognized smart contract vulnerabilities, consistently ranked as a top concern within the blockchain community [2]. Numerous educational resources have been developed to help developers avoid them [11], [12]. Second, the widespread attention has led to diverse static and dynamic detection efforts. Third, reentrancy vulnerabilities often do not act in isolation, but rather in conjunction with other vulnerabilities, amplifying their combined effect. Studying this attack class offers valuable insights into the evolving dynamics between attackers and defenders in blockchain ecosystems, while also helping us to minimize the variability in our analysis.

Most prior studies on reentrancy were conducted before 2020, when few reentrancy attacks had occurred in the wild. Therefore, these studies often rely on incomplete or outdated assumptions. While some recent papers [13], [14] acknowledge more incidents, they lack depth and fail to comprehensively analyze broader trends. Additionally, most papers focus on specific tools or techniques, and do not provide a holistic perspective on reentrancy attacks and their consequences.

In this paper, we present a large-scale empirical study of real-world reentrancy attacks across multiple EVM-compatible blockchains. Our goal is to classify and analyze the types and characteristics of these attacks, while critically examining assumptions from prior research. To this end, we analyze 73 real-world incidents, compiled from industry and community sources, using both qualitative and quantitative approaches. Our qualitative analysis distills insights from notable cases that challenge established views, while our quantitative analysis examines trends across multiple dimensions including blockchain platform, fund loss, reentrancy scope, entry point, attack strategy, vulnerability origin, and attack timeline.

To our knowledge, this is the first comprehensive, in-depth investigation of real-world reentrancy attacks across multiple blockchains. By integrating qualitative and quantitative methods, we: 1) explore diverse attack characteristics, 2) trace the evolution of attacker strategies, 3) expose gaps between academic research and practice, and 4) propose a framework for understanding and classifying reentrancy attacks.

This work thus serves as a vital resource for researchers and developers, providing insights into the evolving threat landscape. Our study's comprehensive approach—combining large-scale data analysis with detailed case studies—yields a unique perspective not found in prior research.

Our key contributions are as follows:

- **Comprehensive Empirical Analysis:** We analyze 73 real-world reentrancy attacks, spanning all publicly documented incidents from 2016 to 2024. This extensive dataset allows us to derive robust insights into various attack characteristics and their evolution.
- **Multi-Dimensional Trend Analysis:** We compile comprehensive statistics on these attacks, uncovering trends across seven critical dimensions: 1) blockchain network, 2) reentrancy scope, 3) entry point, 4) fund loss, 5) attack strategy, 6) vulnerability origin, and 7) attack timeline This multi-dimensional approach provides a holistic view of the reentrancy attack landscape.
- **Novel Attack Classification:** We categorize the attack flow of each incident into distinct classes based on attacker strategies, offering a systematic and novel perspective on reentrancy methodologies.
- **Bridging the Research-Practice Gap:** We critically assess assumptions in existing reentrancy detection research, highlighting discrepancies with real-world attacks and their practical implications. This assessment directly informs our guidelines for future research, ensuring greater relevance and applicability in practice.

Our principal findings are summarized below:

- **Evolved Threats:** Reentrancy attacks have significantly evolved beyond early patterns. Cross-function reentrancy is now prevalent, leveraging diverse entry points (*e.g.*, malicious tokens, ERC hooks) and sophisticated strategies, with price manipulation being the most common. Higher-scope attacks (cross-contract/project/chain) represent a growing, non-negligible threat.
- **Persistent & Rapid Exploitation:** Despite defenses, attacks remain frequent (over 1.5 incidents/month since mid-2021), primarily targeting high-value blockchains. Vulnerabilities are often exploited rapidly, sometimes within days of contract deployment, with attackers increasingly deploying malicious contracts within the exploit transaction itself to evade preemptive detection.
- **Dual Nature & Origins:** The increasing use of reentrancy as an intended design pattern for flexibility complicates detection. However, it is also exploited as a backdoor in malicious schemes (e.g., rug pulls) and vulnerabilities often stem from inheriting flaws via forking popular open-source protocols.
- **Financial Impact Trends:** While individual incident losses vary greatly and can be substantial, the average loss has shown a decline since mid-2022, potentially indicating increased attacker competition or improved (though still imperfect) defensive postures.

II. Background

A. Ethereum and Smart Contracts

Blockchain technology enables decentralized, transparent record-keeping by distributing data across a network of nodes. Ethereum [1], a pioneering blockchain platform, introduced *smart contracts* to support decentralized applications (DApps). Ethereum's success spurred the emergence of other EVM-compatible blockchains, fostering a dynamic ecosystem. Metrics such as *market capacity* (total value of minted cryptocurrencies) and *total value locked* (TVL, assets staked in DeFi platforms) are commonly used to gauge adoption [15].

Smart contracts on EVM-compatible chains are inherently transparent, with their compiled EVM bytecode permanently stored on-chain. Many developers enhance transparency via *source code verification*, publishing source code matched to deployed bytecode, which is accessible through blockchain explorers like Etherscan [16].

B. The DeFi Ecosystem

Tokens are the foundational assets of DeFi, representing various utilities, values, or rights. They are broadly categorized as native tokens (*e.g.*, ETH on Ethereum, POL on Polygon) and standardized tokens such as ERC20 [17], which facilitate fungible asset creation. Other standards—ERC777 [18], ERC721 [19], and ERC1155 [20]—extend support for non-fungible and hybrid tokens [18]–[20].

Exchanges are critical infrastructure for token liquidity and trading. Centralized exchanges (CEXs) are managed platforms matching buyers and sellers via order books, often facilitating fiat-crypto exchanges. In contrast, decentralized exchanges (DEXs) operate without intermediaries, relying on smart contracts and automated market maker (AMM) protocols (*e.g.*, Uniswap, Curve) to determine prices dynamically. This model supports continuous and trustless trading.

Flashloans are another DeFi hallmark, allowing users to borrow assets without collateral, provided repayment occurs within the same transaction. This atomicity enables advanced strategies like arbitrage and liquidation while ensuring lender safety. However, flashloans are also a frequent vector for exploits, including reentrancy attacks and price manipulation.

```
1  contract Victim {
2      mapping(address => uint256) public blc;
3      function withdraw(uint256 _amount) public {
4          require(_amount <= blc[msg.sender]);
5          // Transfer ETH to the caller
6          msg.sender.call{value: _amount}();
7          // Update the caller's balance
8          blc[msg.sender] -= _amount;
9      }
10 }
11 contract Attacker {
12     Victim private vic;
13     address private owner;
14     // Used to initiate the attack
15     function attack() public {
16         require(msg.sender == owner);
17         vic.withdraw(vic.blc(address(this)));
18         owner.transfer(this.balance);
19     }
20     fallback() external payable {
21         uint256 v = vic.blc(address(this));
22         if (address(vic).balance >= v)
23             // Reenter victim to drain its ETH
24             vic.withdraw(v);
25     }
26 }
```

Listing 1: Reentrancy example

C. Reentrancy Attacks

Reentrancy attacks constitute a significant and well-documented class of vulnerabilities in smart contracts. They were ranked as the top vulnerability of smart contracts as per the OWASP classification until 2024, and are in the top five vulnerabilities in 2025 [2], [3].

Listing 1 provides a concrete example of a reentrancy vulnerability and its exploitation. In this instance, the `Victim` contract updates the caller's balance *after* transferring the requested Ether (ETH) via the `call` function. This sequence triggers the `fallback` function within the `Attacker` contract *before* the balance update in `Victim` is finalized, leading to a state inconsistency. By exploiting this inconsistency, the attacker can repeatedly invoke `Victim.withdraw` to illicitly drain the `Victim` contract's ETH reserves.

To be more specific, the defining pattern of a reentrancy attacks can be formalized by the following formula:

$$V_1.f_{V_1} \rightarrow A.f_A \rightarrow V_2.f_{V_2} \tag{1}$$

where V_1 and V_2 represent victim contracts, A denotes the attacker's contract, f represents functions within the respective contracts, . operator denotes membership relation, and the arrow ($\rightarrow$) signifies an invocation relationship, either direct or indirect. In essence, a reentrancy attack occurs when a vulnerable contract (V_1) calls a function (f_A) in an attacker's contract (A), which in turn calls back into a function (f_{V_2}) within a victim contract (V_2) before the initial execution ($V_1.f_{V_1}$) completes.

A common classification scheme for reentrancy attacks is based on their *scope*, a metric that characterizes the relationship between f_{V_1} and f_{V_2} in Formula 1. This includes:

- **Single-function reentrancy**: $V_1 = V_2$, $f_{V_1} = f_{V_2}$; the same function is reentered.
- **Cross-function reentrancy**: $V_1 = V_2$, $f_{V_1} \neq f_{V_2}$; distinct functions in the same contract.
- **Cross-contract reentrancy**: $V_1 \neq V_2$; reentry targets a different contract.

We extend this classification to capture emerging behaviors:

- **Cross-project reentrancy**: $V_1 \neq V_2$, and the contracts belong to different projects.
- **Cross-chain reentrancy**: The invocation chain spans multiple blockchain networks.

Beyond the scope-based classification, another distinct category is **read-only reentrancy**, where f_{V_2} is a `view` function that only reads the contract state without modifying it. Such invocations are benign by themselves, but they may influence logic in subsequent operations, leading to evasive reentrancy attacks that are harder to detect.

III. Related Work

The significant fund loss from The DAO attack in 2016 first brought reentrancy vulnerabilities into the spotlight [21]. Researchers have since proposed various detection methods, including static analysis, dynamic analysis, and, more recently, attacker-focused approaches [22].

Static Analysis: These tools detect vulnerabilities pre-deployment. Early tools like Oyente [7] and Manticore [23] used symbolic execution. Slither [9] introduced an intermediate representation (IR) for Solidity, enabling broader detection. Others like Zeus [24], Securify [25], and Vandal [26] applied formal verification with predefined policies. While effective in specific domains, most static tools struggle with cross-contract interactions (common in DeFi, *e.g.*, via proxy contracts [27]) and suffer from high false positive/negative rates [8].

Dynamic Analysis: These tools monitor execution to detect vulnerabilities at runtime. For instance, Sereum [10] uses taint analysis integrated into Ethereum to flag suspicious behavior. ReGuard [28] and ContractFuzzer [29] use fuzzing to generate inputs and examine traces. While Sereum handles cross-contract interactions, it only supports post-deployment detection. Fuzzers, though effective, face scalability challenges with complex input formats [30].

Recent Advancements: Recognizing existing tool limitations, recent research explored novel approaches. Roychoudhury *et al*. [13] analyzed attacker contract patterns. However, many recent exploits deploy attack contracts mid-transaction, preventing preemptive detection (see Section VI-G). Zhang *et al*. [31] and Grundy *et al*. [32] proposed tools for analyzing cross-contract interactions. Despite advances, comprehensive and proactive reentrancy detection remains unsolved.

Additionally, recent benchmarks on real-world and synthetic datasets expose significant shortcomings in existing tools. Ghaleb *et al*. [8] assessed six widely-used static analyzers on synthetic contracts, revealing substantial false positive/negative rates. Similarly, Zheng *et al*. [14] evaluated five major tools on real Ethereum transactions, finding high FP rates (up to 99.8%) and failures to detect non-classic reentrancy patterns (*i.e.*, deviating from `call.value`). These results align with our observations, underscoring the need for more robust reentrancy detection mechanisms.

IV. Methodology

This study employs an empirical analysis conducted through independent manual investigation by two researchers, both of whom are co-authors of this paper. The analysis comprises five stages: 1) collection of information on real-world reentrancy attacks from online sources; 2) extraction of transaction metadata to characterize each incident; 3) independent inspection of invocation flows to identify vulnerabilities and attack mechanisms; 4) clustering and categorization of incidents based on attack flow similarities; and 5) reconciliation of results through discussion to resolve discrepancies. We describe each step.

A. Data Collection

We collect reentrancy incidents from three primary sources: 1) a GitHub repository curated by a white-hat researcher [4], 2) BlockSec's incident records [5], and 3) SlowMist's hack events archive [6]. Although these sources are neither exhaustive nor verified, they are the most comprehensive public datasets available, comprising 80 incidents. We exclude seven cases: six that we determined do not involve reentrancy upon

further analysis, and one on a lesser-known blockchain unsupported by our tooling. This yields a dataset of 73 incidents. To support reproducibility and further research in this area, the curated dataset used in this study has been made publicly available [33].

These sources provide key details such as victim project names, dates, and often links to original reports by security firms, post-mortems, or social media disclosures. When available, transaction hashes are used to locate the exploit transactions. We supplement this data with targeted searches for public reports and verify each incident by inspecting the attacker's address on Etherscan.

B. Metadata Extraction

We extract relevant metadata from each exploit transaction, including the timestamp of the initial transaction, the involved blockchain(s), and the deployment timestamps of both vulnerable and attacker contracts. These details enable us to analyze execution patterns and track the timeline from vulnerability introduction to exploitation.

C. Transaction Analysis

We use BlockSec Phalcon Explorer [5] and Tenderly Explorer [34] to examine token flows and call traces. The call trace presents inter-contract EVM operations as a stack of nested calls, helping us identify the reentrancy entry point, scope, and vulnerability origin using verified contract source code from Etherscan. Token flow analysis corroborates the findings and quantifies financial impact.

Although public incident reports are referenced, they often lack sufficient detail or contain inconsistencies. Therefore, the insights in this study are derived from our own analyses.

D. Categorization

Following transaction analysis, we adopt an open coding approach to cluster incidents based on similarities in attack flows. We first define initial codes reflecting specific exploit structures, then generalize them by merging those with shared underlying strategies. For example, attacks involving price oracle manipulation and pool formula manipulation are grouped under the broader category of "price manipulation." We prioritize alignment with established terminology where possible. However, many clusters do not align well with existing terms. Consequently, we introduce new terminologies to describe these emergent categories.

E. Cross-Validation

Upon completing independent analyses, the two researchers cross-review each other's findings to identify and resolve discrepancies. Any disagreements are discussed until consensus is reached. This collaborative validation minimizes interpretation bias and ensures consistency across cases.

V. Qualitative Analysis

We conducted a qualitative analysis of reentrancy attacks and extracted high-level insights and characteristics. Each case study highlights one or more attacks that illustrate these insights and characteristics.

A. Recurrent Simple Pattern

Though often viewed as a classic or outdated vulnerability, reentrancy remains a significant threat, with its simplest form still exploited in modern DApps. Since first appearing on Ethereum in 2016, such attacks have typically followed a straightforward pattern: the victim contract uses `call.value` to transfer ETH to the caller, triggering the caller's `fallback` function before updating the balance. While techniques have grown more sophisticated over time, basic variants of this attack continue to appear.

Case Study 1: On February 12, 2024, a reentrancy attack on the Web3 game ChainPaint resulted in a $52.5K ETH loss. The vulnerability resided in the game's auction system, where participants updated bids via `Auction.makeBid`. This function first refunded the previous bid using `call.value` , then updated the bid to the new amount. By placing a higher bid and exploiting their contract's `fallback` function, the attacker recursively reentered `Auction.makeBid`, draining all accumulated bids.

B. Callback as a Design Feature

While most victim contracts inadvertently invoke attacker-controlled functions, an increasing number of recent projects explicitly provide custom callback hooks—either for specific business logic or programming flexibility. However, such practices can lead to false positives in existing detection tools [35].

A prominent example is flashloan. Although reentrancy into a flashloan provider's contract is generally unintended, the business logic of flashloans inherently requires transferring control to the caller, making external calls a core part of the process. To mitigate this risk, most flashloan providers implement reentrancy guard. Nevertheless, some developers neglect this critical measure, leaving contracts vulnerable.

Case Study 2: Peapods Finance (December 23, 2023), 0x0 (October 27, 2023), and DFX Finance (November 10, 2022) suffered reentrancy attacks due to unprotected flashloan functions. These contracts lacked effective reentrancy guards and verified repayments merely through balance checks. Exploiting this flaw, attackers *deposited* the borrowed tokens back into the contract during reentrancy, rather than *repaying* them. As a result, the contracts both credited the attackers' accounts and cleared their debts—allowing the same tokens to serve as both repayment and deposit, enabling free withdrawals.

Some developers have taken this practice further, not only providing custom callback hooks but also designing functions intended to be reentered to accommodate their business logic.

Case Study 3: On May 14, 2024, Predy Finance was exploited in a reentrancy attack, losing $460K in tokens. This case illustrates the risks of mismanaging reentrancy as a feature. Predy Finance introduced a custom callback hook, `predyAfterTradeCallback`, invoked after token transfers in `PredyPool.trade`. Additionally, it offered `PredyPool.take`, a deliberately reenterable function allowing arbitrary token transfers. Although intended for legitimate use, inadequate safeguards on token pair registration allowed attackers to target and exploit tokens belonging to other users.

TABLE I: Case Study Summary for Qualitative Analysis

No.	Name	Date	Vulnerability Description
1	ChainPaint Attack	Feb 12, 2024	Old-school pattern using user balance updates after `call.value` transfers, similar to the vulnerability exploited in The DAO attack.
2	Peapods Finance Whitehat Attack	Dec 13, 2023	Flashloan functions lacked reentrancy guards and checked repayment by contract balance only, enabling attackers to register deposits made in reentrancy as repayment.
	0x0 Attack	Oct 27, 2023	
	DFX Finance Attack	Nov 10, 2022	
3	Predy Finance Attack	May 14, 2024	Reentrancy used as a feature, allowing callers to reenter `PredyPool` via a custom callback function and move funds with the `take` function. But the failure to check each individual pair's balance led to a successful attack.
4	Revest Finance Attack	Mar 27, 2022	Stale `FNFTHandler.fnftsCreated` variable enabled reentrancy during minting, allowing attackers to overwrite data of newly minted FNFTs.
5	Omni Protocol Attack	Jul 10, 2022	Bypassed collateral checks in `withdrawERC721` by manipulating configurations through a reentrant call, allowing withdrawal of NFTs still used as collateral.
6	BNB Brokers Rug Pull	Apr 27, 2022	Suspected rug pull: used reentrancy backdoor to drain the contract's funds, followed by deactivation of social media and shutdown of project websites.
	Barley Finance Rug Pull	Jan 28, 2024	
7	GoodDollar Attack	Dec 26, 2023	Public accessibility of an privileged function allowed the attacker to exploit reentrancy to simulate interest collection, causing artificial appreciation of G$X tokens.
8	Curve Attack	Jul 30, 2023	Bug in the Vyper compiler invalidated its build-in reentrancy guard, rendering Curve vulnerable to price manipulation reentrancy attack.
9	Terra Attack	Jul 31, 2024	Mishandled timeout message in Terra's IBC implementation enabled attackers to mint additional legitimate tokens at no cost.

C. Indirect State Variables

A universal trait of reentrancy attacks is the manipulation of state variables before they are properly updated. These typically involve accounting variables such as balances, debts, or token supply. As a result, developers often focus on securing these financial states. However, in some cases, variables that appear unrelated can indirectly affect accounting logic through subtle contract interactions, leading to significant losses.

Case Study 4: On March 27, 2022, Revest Finance was exploited in a reentrancy attack, losing $2.01M. The vulnerable variable was not tied to balances but tracked the number of minted NFTs (`FNFTHandler.fnftsCreated`). This ID counter is incremented only after minting is complete. The attacker exploited reentrancy through `onERC1155Received`—triggered after the FNFT was transferred—while the counter remained stale. The functions involved, `Revest.mintAddressLock` and `Revest.depositAdditionalToFNFT`, are designed to mint new FNFTs and allow users to deposit additional tokens to an existing FNFT, respectively. During reentrancy, the stale counter caused the same ID to be assigned, overwriting the FNFT's data—including associated RENA token amounts—and resulting in substantial loss.

Case Study 5: On July 10, 2022, Omni Protocol suffered a $1.4M exploit. As with Revest, the critical variable was not a direct financial balance. The vulnerability lay in `Pool.withdrawERC721`, which returns ERC721 NFTs supplied by users. NFTs could also be used as collateral to borrow tokens, so `SupplyLogic.executeWithdrawERC721` validated that users retained sufficient collateral post-withdrawal. This check, however, occurred after the NFT transfer and relied on `userConfig.isUsingAsCollateral`. Because ERC721 includes the `onERC721Received` post-transfer hook, the attacker used it to invoke `Pool.liquidationERC721`, manipulating the collateral status and bypassing validation, leading to unauthorized withdrawals eventually.

D. Reentrancy for Rug Pulls

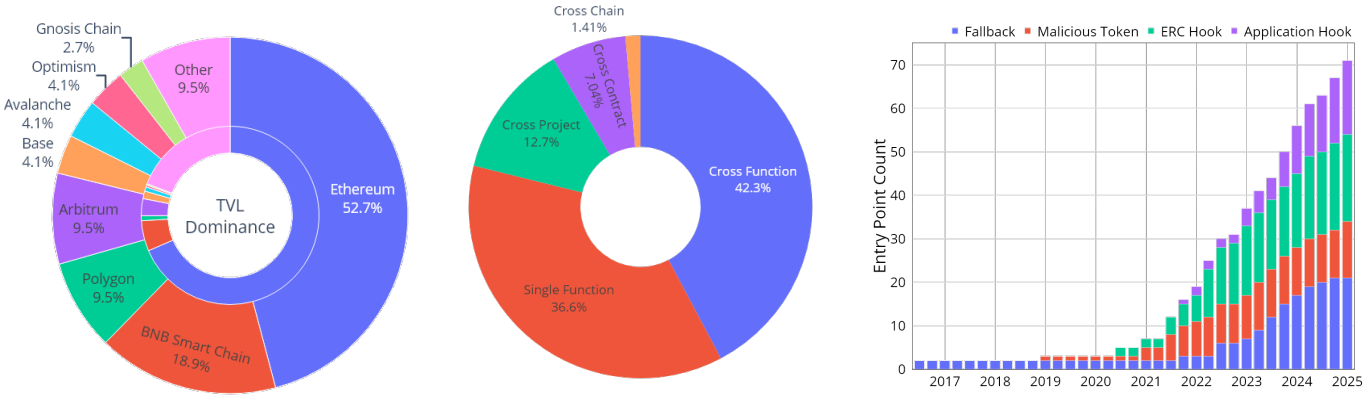
While most reentrancy attacks are attributed to external hackers, we suspect that some are orchestrated by smart contract developers themselves. "Rug pull" is an act by developers in which they misappropriate users' investments [36].

Although it is often difficult to conclusively classify an incident as a rug pull, some cases raise strong suspicion. Two notable examples suggest that developers may have deliberately introduced reentrancy vulnerabilities—an assessment corroborated by other blockchain security firms.

Case Study 6: BNB Brokers and Barley Finance were attacked on April 27, 2022, and January 28, 2024, respectively. The BNB Brokers exploit followed the pattern of The DAO hack, while Barley Finance suffered from an unprotected flashloan vulnerability. In both cases, the projects' social media accounts were abruptly deleted, their websites taken offline, and no further transactions occurred on the affected contracts. These suspicious behaviors suggest premeditated rug pulls by the developers [37], [38].

E. Insufficient Access Control

As the DeFi ecosystem evolves, inter-contract interactions and modular programming practices have become increasingly common. To facilitate manual administration and interactions with other contracts, certain functions are often declared as `public` or `external`. While many developers implement whitelist mechanisms to restrict access to sensitive administrative functions, others either overlook this critical safeguard or opt for greater flexibility at the expense of security. Consequently, some contracts become vulnerable to reentrancy attacks with impersonation strategies (see VI-E).



(a) Blockchain network (b) Reentrancy scope (c) Entry point

Fig. 1: Statistics of blockchain network, reentrancy scope, and entry point

Case Study 7: On December 26, 2023, GoodDollar experienced a sophisticated cross-contract price manipulation attack, resulting in a $630K loss. The attacker targeted the `GoodFundManager.collectInterest` function, originally intended for developers to collect interest from staking the contract's DAI balance in pre-configured farms. This interest would be deposited into Compound to obtain cDAI, which would then be added to GoodDollar's pool, minting G$X tokens. While the process was designed to slightly appreciate G$X, the effect was minimal under normal conditions. However, because `collectInterest` was declared public, it was accessible to anyone, including the attacker. Through reentrancy, the attacker deposited a large amount of cDAI, deceiving the contract into treating it as interest income—artificially inflating G$X's value. The attacker then sold G$X tokens acquired before the manipulation to realize the profit.

F. Vulnerable Infrastructure

While reentrancy vulnerabilities often arise from flaws in smart contracts, such as the victim contracts or their dependencies, they can also stem from bugs in the infrastructure.

Case Study 8: On July 30, 2023, Curve, one of Ethereum's leading DEXs, was exploited in a reentrancy attack that resulted in $60.7M in losses [39]. The attacker invoked `add_liquidity` during the execution of `remove_liquidity`, before the attacker's balance and the pool's total supply were updated—a tactic seen in early attacks on Curve-like protocols. Notably, Curve's developers had implemented reentrancy guards for these functions, but a bug in the Vyper compiler rendered them ineffective. This vulnerability coincided with Curve's deployment of an upgraded contract compiled using the affected Vyper version [40].

Case Study 9: On July 31, 2024, the Terra Network suffered a $4.58M reentrancy exploit [41]. The vulnerability was rooted in Terra's implementation of the Inter-Blockchain Communication (IBC) protocol, which enables cross-chain interactions such as wrapped token bridging. The attacker recursively triggered `MsgTimeout` to exploit a flaw in the protocol's timeout logic, causing the blockchain to mint additional tokens without asset backing. This incident remains the only documented example of cross-chain reentrancy.

VI. Quantitative Analysis

This section presents a quantitative analysis of the 73 reentrancy attacks collected (sources detailed in Section IV-A). The analysis focuses on seven key dimensions, chosen for consistent data availability and relevance to understanding attack characteristics[1]: 1) blockchain network, 2) reentrancy scope, 3) entry point, 4) fund loss, 5) attack strategy, 6) vulnerability origin, and 7) attack timeline. We chose these dimensions to maximize the comprehensiveness of the analysis.

A. Blockchain Network

As of this writing, 1,136 EVM-compatible blockchains exist [43], with seven networks each boasting a Total Value Locked (TVL) exceeding $1 billion [15]. This number continues to grow. Figure 1a illustrates the distribution of reentrancy attacks across blockchain networks (outer ring)[2], alongside the relative TVLs of those chains (inner ring).

Despite blockchain proliferation, reentrancy attacks remain predominantly concentrated on Ethereum (52.7% of documented incidents). BNB Smart Chain, Polygon, and Arbitrum are also frequent targets (18.9%, 9.5%, and 9.5%, respectively). This distribution strongly correlates with network TVL, supporting the hypothesis that attackers prioritize high-value targets for greater potential financial rewards. We also examined the total number of contracts deployed [44] per blockchain but found no correlation with attack frequency.

Key Findings: Blockchain Network

Reentrancy attacks are heavily concentrated on high-value networks, particularly those with substantial TVL. Ethereum alone accounts for more than half (52.7%) of all documented incidents.

B. Reentrancy Scope

Attack scope refers to the functional boundaries involved in the exploit (see Section II-C). Figure 1b illustrates the

[1]Not all dimensions include data for every incident. *e.g.*, the Iron Finance attack [42] lacked a corresponding reentrancy transaction.

[2]The sum of the percentages exceeds 100% because some attacks occurred on multiple chains.

distribution of attacks by scope. Early attacks were predominantly single-function, but as defensive mechanisms evolved, attackers increasingly adopted broader scopes. This shift is evident as cross-function reentrancy has surpassed single-function reentrancy, now accounting for 42.3% of recorded attacks, compared to 36.6% for single-function incidents. Conversely, cross-contract and cross-project reentrancy, despite their academic prominence, constitute only 21.1% of cases.

These findings also underscore a critical gap in developer awareness, as single-function and many cross-function vulnerabilities can be effectively mitigated using reentrancy guard (see Section VII-B).

Key Findings: Reentrancy Scope

Cross-function and single-function reentrancy attacks dominate the landscape, comprising 42.3% and 36.6% of all recorded attacks respectively. Comparably, higher-scoped reentrancy attacks are notable but less common.

C. Entry Point

The entry point of a reentrancy attack refers to the function within the attacker's contract that is invoked by the victim contract during exploitation. This function typically contains the core reentrancy logic.

Based on the function's nature and associated token standards, we classify them into four types:

1) **Fallback:** The contract's default function, triggered when no function signature is specified or matched.
2) **Malicious Token:** Standard ERC-20 functions (*e.g.*, `transfer`, `transferFrom`, `approve`) within custom tokens controlled by attackers.
3) **ERC Hook:** Callback functions defined by standards such as ERC-721, ERC-777, and ERC-1155, automatically invoked before or after token transfers.
4) **Application Hook:** Non-standard callback functions implemented by specific projects to support custom logic.

The use of these entry points has evolved over time, as shown in Figure 1c. Early attacks mainly exploited the `fallback` function, typically triggered during native token transfers via `call.value`, `transfer`, or `send`. Among these, only `call.value` enables reentrancy, due to the 21,000 gas limitations imposed on the other two. Consequently, most early detection efforts focused on this pattern, which has since been recognized as a limitation [13], [14]. Although the `fallback` remains relevant (see Section V-A), attackers have increasingly employed malicious tokens since around 2020, especially against platforms supporting arbitrary token registration. In parallel, ERC-defined transfer hooks became popular vectors. By late 2022, application-specific hooks had emerged as a notable trend.

Overall, `fallback` and ERC hooks are the most frequently exploited, accounting for 29.6% and 28.2% of cases, respectively. Application hooks and malicious tokens make up 24.0% and 18.3%, respectively.

Key Findings: Entry Point

Reentrancy attack entry points have diversified over time—from `fallback` functions to malicious tokens, ERC hooks, and project-specific application hooks. Despite this shift, `fallback` and ERC hooks remain the most dominant, each involved in nearly 30% of incidents.

D. Fund Loss

Fund loss refers to the market value of stolen tokens at the time of the attack (stolen amount × token price). While a practical approximation, this estimate doesn't capture full victim impact (*e.g.*, token depreciation, loss of investor confidence) nor does it equal attacker profit, which is reduced by costs (*e.g.*, DEX fees, flashloans) and liquidity constraints when converting large amounts of stolen tokens.

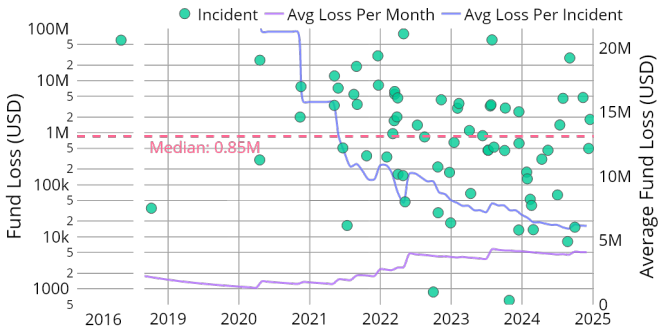


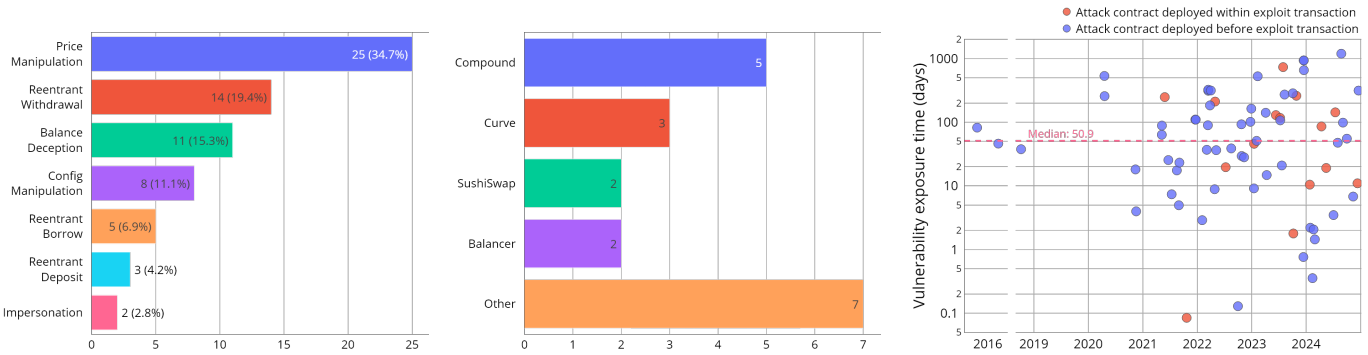
Fig. 2: Fund loss

Figure 2 illustrates the fund losses from reentrancy attacks over time (white-hat attacks are excluded from this figure). Among the analyzed incidents, losses range from under $1K (0x0 attacks) to nearly $80M (Fei Protocol attack). For comparison, the DAO attack caused losses of around $60M. However, due to the Ethereum hard fork that ensued, the attacker's realized profit was substantially lower [21].

Despite the ongoing prevalence of reentrancy attacks on EVM-compatible blockchains, the average fund loss per incident has exhibited a steady downward trend since their emergence. Similarly, aggregate monthly losses plateaued and began declining after mid-2022. A brief spike in mid-2023—primarily due to an exceptionally high-value attack on Curve [39]—does not contradict the overall downward trajectory. While the precise causes of this decline remain uncertain, likely contributing factors include: 1) increased competition among attackers, 2) improved security awareness among developers, and 3) the rapid expansion of the smart contract ecosystem, which may dilute the average market value per project.

Key Findings: Fund Loss

Reentrancy attacks result in highly variable losses, from under $1K to $80M. Since mid-2022, average losses have steadily declined, indicating a notable reduction in the financial impact of these attacks over time.



(a) Attack strategy (b) Vulnerability origin (c) Attack timeline

Fig. 3: Statistics of attack strategy, vulnerability origin and attack timeline

E. Attack Strategy

Researchers have paid limited attention to the strategies employed by attackers. While Huang *et al*. identified common patterns in attack contracts [13] and Chen *et al*. classified real-world attacks into three categories [14], these studies do not provide a comprehensive examination of attack flows.

In our analyses, we identified seven distinct strategies, offering a more granular understanding of attackers' methodologies and approaches. These strategies represent key insights into how the attackers have profited from the vulnerabilities. Figure 3a shows the distribution of reentrancy attacks based on the attack strategy. The three most dominant strategies comprise 74.3% of all attacks, while strategies like *Impersonation* only account for few cases. Details of each strategy are described below, in decreasing order of their prevalence.

Price Manipulation: Similar to market manipulation in traditional finance, blockchain speculators seek to profit by influencing token prices. While standard market tactics often require multiple trades over time, reentrancy vulnerabilities provide attackers with a streamlined approach. Leveraging flashloans to amass the necessary capital and exploit such vulnerabilities in a single transaction, the attackers can bypass the usual risks of speculative trading. Most price manipulation attacks exploit stale variables involved in price calculations, resulting in dramatic token value swings. For example, during the Jaypeggers attack, the JAY token depreciated by 99.3% mid-attack [45], a level of manipulation rarely seen in traditional markets. Notably, price manipulation reentrancy was the sole strategy in all cross-project reentrancy cases.

Reentrant Withdrawal: Reentrant withdrawal occurs when attackers recursively withdraw funds deposited in a target contract, typically due to the failure to implement the checks-effects-interactions pattern (see Section VII-B). Although the frequency of reentrant withdrawals has declined in recent years, as discussed in Section V-A, the strategy remains relevant. It accounts for 14 attacks in our dataset and is the second most common reentrancy technique.

Balance Deception: In this strategy, attackers exploit insufficient balance verification mechanisms in DeFi contracts. Post-transfer balance checks are manipulated to deceive contracts into believing conditions have been met, allowing attackers to siphon funds. One common scenario is flashloan providers failing to implement reentrancy guard on their deposit functions. Consequently, attackers can deposit the loan back to the providers in reentrancy and mislead the contracts into believing that they have received the repayment, as illustrated in Section V-B. Another notable instance of this strategy is the Lendf.Me attack [46], where `MoneyMarket` calculates the new balance before the transfer but uses it after its `supply` function, allowing the attacker to `withdraw` during reentrancy to reclaim their deposits but still receiving credits.

Configuration Manipulation: Many DeFi applications employ on-chain configurations to implement custom business logic. However, improper safeguarding of these configuration values can expose them to reentrancy attacks. For instance, two NFT projects, CreatureToadz [47] and HypeBeans [48], were exploited on October 21, 2021, and February 3, 2023, respectively. Both projects featured a pre-sale phase that allowed specific users to mint one NFT per wallet. Attackers utilized the `onERC721Received` hook to mint recursively, bypassing the one-per-wallet restriction. Another example is the Revest Finance attack [49], where the attacker exploited an ERC1155 hook to overwrite an NFT's configuration data, including the associated amount of RENA (see Section V-C).

Reentrant Deposit: Although conceptually the opposite of reentrant withdrawal, attackers can also use recursive deposits to their advantage. The vulnerability arises when a target contract calculates user balances by measuring differences before and after token transfers. By recursively depositing during reentrancy, attackers accumulate multiple credits for a single deposit, allowing them to withdraw more tokens than they originally deposited. Example incidents include Paraluni attack [50] and Grim Finance attack [51].

Reentrant Borrow: Similar to reentrant withdrawal, reentrant borrow attacks involve exploiting lending protocols to bypass borrowing limits. These attacks are distinct in that attackers also aim to redeem their collateral after repayment. A representative example is the DeFiPie attack [52]. In this case, the attacker first created malicious tokens and registered them as eligible collateral in the protocol. Next, they created token pairs on Uniswap, which acted as the protocol's price oracle, thereby assigning these malicious tokens a market

price. Finally, the attacker used these tokens to repay loans and redeem collateral. Notably, all three reentrant borrow victims analyzed—including DeFiPie—were forks of Compound, inheriting its vulnerabilities (see Section VI-F).

Impersonation: An impersonation attack occurs when a contract fails to secure critical functions with proper access controls like whitelist mechanisms. A notable example is the Abracadabra Money white-hat attack [53]. Here, the `CauldronV4` contract supported cloning and invoked the `init` function to set an oracle address. During this process, the oracle's `get` function was called. However, this function lacked proper access control, allowing an attacker to create a malicious oracle contract. By reentering `CauldronV4`, the attacker could transfer other users' tokens to themselves. Fortunately, the developer team identified the vulnerability before it could be exploited by attackers, and preemptively executed the attack themselves, transferring users' funds to a more secure contract.

Key Findings: Attack Strategy

We found seven strategies employed by attackers. **Price Manipulation** is the most prevalent, enabling attackers to inflate or deflate token prices. **Reentrant Withdrawal** remains a widely studied approach, though its occurrence has decreased recently. **Balance Deception** targets flawed balance checks, allowing attackers to bypass repayment and credit constraints. Together, these three strategies account for 74.3% of all reentrancy attacks analyzed.

F. Vulnerability Origin

While most victims of reentrancy attacks suffer from flaws in their own code, vulnerabilities can also originate externally, such as through forked codebases or interactions with other smart contracts (*e.g.*, price oracles). Our analysis identifies eleven examples of external vulnerabilities. Figure 3b depicts the distribution of reentrancy vulnerabilities by their origins (projects with only a single occurrence are grouped under "Other"). Below, we describe the four sources that were linked to more than one incident.

Compound: A foundational lending protocol on Ethereum. Compound's codebase has been widely adopted by numerous DeFi projects. However, earlier versions of Compound contained reentrancy vulnerabilities, which were patched on June 6, 2022. Compound itself avoided exploitation due to its rigorous vetting process, which limits interactions to only standard ERC20 tokens. However, projects that forked older versions of Compound became susceptible to attacks.

Curve: A prominent DEX and AMM on Ethereum. Curve's price oracle, widely used by other projects, contained a read-only reentrancy vulnerability. Although Curve itself remained unaffected, projects relying on this price feed became vulnerable to price manipulation.

SushiSwap: A well-known Uniswap fork. SushiSwap introduced the `MasterChef` contract, adopted by many DeFi projects. Unfortunately, the `MasterChef.emergencyWithdraw` function contains a clear reentrancy vulnerability, enabling attackers to execute reentrant withdrawals and drain funds.

Balancer: Another well-established AMM protocol on Ethereum. But just like Curve, its widely used price oracle was affected by a read-only reentrancy vulnerability, exposing dependent projects to price manipulation attacks.

Key Findings: Vulnerability Origin

Some projects inherit reentrancy vulnerabilities from popular protocols like Compound and SushiSwap, or face exploitation due to read-only vulnerabilities in external price oracles like Curve and Balancer. This highlights the risks of forking codebases or relying on external contracts.

G. Attack Timeline

The timeline of an attack provides valuable insights into its progression. From the attacker's viewpoint, the process is relatively straightforward: identifying a vulnerability in a deployed contract, developing an exploit strategy, deploying an attack contract, and executing the exploit transaction. We take the attacker's viewpoint. Our analysis of timestamps for vulnerable contract deployments, attack contract deployments, and first exploit transactions reveals that vulnerability exposure time—the interval between a vulnerable contract's deployment and the first exploit transaction—is *relatively short*. The median exposure time is 50.9 days, with extreme cases occurring within a single day. This is surprising given the sophistication of many attacks, which often take days for blockchain security companies to analyze and mitigate post-incident. This suggests two possibilities: either attackers possess exceptional skills for rapid vulnerability discovery and exploitation, or vulnerabilities are publicly accessible before deployment through channels such as GitHub repositories. The latter seems more plausible.

Another observation is the evolving strategy of attackers. Increasingly, attackers deploy their malicious contracts directly within exploit transactions (highlighted in red in Figure 3c), making preemptive detection impossible. This evolution has significantly reduced the efficacy of methods like BlockWatchdog [13] and Backrunner [54], which aim to identify attackers' contracts before exploitation.

Key Findings: Attack Timeline

Attackers often exploit vulnerabilities within days after contract deployment, sometimes even hours, with a median exposure time of 50.9 days. Attackers increasingly deploy malicious contracts within exploit transactions, bypassing preemptive detection mechanisms.

VII. Discussion

We first examine common misconceptions about reentrancy attacks, followed by an evaluation of two prevalent mitigation techniques. We then propose research guidelines based on our findings, and explore additional perspectives, including

TABLE II: Myths and Realities about Reentrancy Attacks, plus the Implications for Detection Techniques

Section	Myth	Reality	Explanation
VI-C VI-D VI-G	They decrease because of the wide adoption of reentrancy guard [55].	They remain prevalent to this day of writing.	After the DAO attack, reentrancy attacks ceased for over two years. Following the SpankChain attack (Oct 19, 2018) [56], another 1.5 years passed without incidents. However, reentrancy attacks surged in mid-2021 and have persisted at a rate of 1.5 incidents per month on average ever since (see Figure 1c, 2, 3c).
VI-C	They only target the `fallback` function [57].	They utilize various entry points [58].	While the DAO attack used the `fallback` function, modern attacks target malicious tokens, ERC hooks, and application-specific hooks. The `fallback` function accounts for only 30.4% of analyzed attacks.
VI-C	They only exploit native tokens and fungible tokens [59], [60].	They also target other types of tokens, including NFTs and hybrid tokens.	Although most attacks involve fungible tokens, NFTs such as CreatureToadz [47] and HypeBeans [48], along with hybrid tokens [49], have also been exploited.
VI-E	They all involve recursive withdrawal [57].	Reentrant (recursive) withdrawal is only one of many strategies attackers used.	The first two reentrancy attacks before 2020 did use reentrant withdrawal. However, the third attack (imBTC on April 18, 2020 [61]) applied a different one: price manipulation. Our analysis identifies seven distinct attack strategies.
V-B	Reentrancy is inherently malicious [62].	Reentrancy can also be legitimate.	Reentrancy is a common pattern that could be used to support modular and flexible designs. This is increasingly prevalent with the rise of cross-contract interactions, especially after the adoption of proxy contracts for upgradability.

reentrancy on non-EVM-compatible blockchains, applicability of our methodology to other attacks, and a victim-centric view of vulnerabilities. Finally, we outline the threats to the validity.

A. Reentrancy Attacks: Myths and Reality

Our qualitative and quantitative analyses reveal persistent misconceptions about reentrancy attacks in both academic and industrial contexts. In research, these misconceptions lead to limited detection efficacy in real-world scenarios; in practice, they may result in design flaws that expose contracts to exploitation. Table II summarizes these myths and the corresponding realities. However, many misconceptions do not stem from negligence. Most widely cited static analysis tools were developed before 2020, when only a few real-world attacks had occurred. These tools could not have anticipated the sophisticated strategies adopted by modern attackers.

B. Mitigation Techniques

The two most commonly adopted techniques to mitigate reentrancy attacks are the checks-effects-interactions (CEI) pattern [12] and reentrancy guard [63]. However, both approaches have inherent limitations as we describe below.

Reentrancy Guard: This method applies a modifier that reverts transactions when protected functions are recursively invoked. It is effective for single-function reentrancy but less so for cross-contract or higher-scoped cases. Its protection scope depends on implementation: guards deployed in a shared base contracts offer broader coverage than those directly embedded within single Solidity contracts[3]. Our quantitative analysis shows that while 65.8% of incidents could be prevented by reentrancy guards, this rate drops to 51.4% after 2023. Notably, cross-contract and higher-scoped attacks, though less frequent, account for 57.2% of total fund losses.

CEI Pattern: The CEI pattern advises developers to update state variables (effects) before making external calls (interactions), ensuring that the contract's state remains consistent when the execution flow is transferred to other contracts. This is the most common approach to address reentrancy vulnerabilities after an attack. However, in some cases, the contract's business logic necessitates using state differences before and after interactions, rendering the CEI pattern impractical.

[3]An on-chain contract with a unique address is typically linked from multiple contracts in Solidity source code.

C. Guidelines for Future Research

The discrepancies between common misconceptions and the observed realities of reentrancy attacks, as revealed through our analyses, highlights several critical directions for future research in this domain. We outline three key areas that demand renewed attention.

First, the persistent prevalence of reentrancy attacks underscores a continuing arms race between attackers and defenders (developers, auditors, and researchers). The diminishing effectiveness of established mitigation techniques—evidenced by a decrease in the proportion of attacks preventable by reentrancy guards from 81.6% before 2023 to 51.4% after 2023—further emphasizes the continued need for ongoing research and development in reentrancy prevention.

Second, the evolution of attack strategies has rendered many early assumptions obsolete—particularly those published before 2020. Future work must move beyond fixed entry points such as the `fallback` function and avoid presuming which token standards are likely to be targeted. Reentrancy now spans a broad range of token types and frequently co-occurs with other vulnerabilities, including access control flaws (see Section V-E) and over-reliance on external price oracles [64]–[66]. Researchers should move on from viewing reentrancy not just as a means of draining funds through recursion, but also as a mechanism for exploiting broader security vulnerabilities.

Third, the increasing adoption of reentrancy as a deliberate programming pattern to enhance code flexibility warrants careful consideration. This trend blurs the purpose of reentrancy, transforming it from a clear indicator of vulnerability to a more ambiguous characteristic. This shift necessitates that future tools distinguish between malicious exploits and legitimate uses of reentrancy to avoid false-positives.

D. Reentrancy on Non EVM-Compatible Blockchains

While most blockchains adopt EVM compatibility for Ethereum interoperability, some platforms introduce new architectures to address EVM limitations, such as high transaction fees, low throughput, and high latency. Solana and EOS are two notable examples.

Solana, often referred to as an "Ethereum killer", has made significant strides in transaction speed, fee optimization, and scalability [67]. To mitigate the risk of reentrancy attacks, Solana prohibits recursion into the same program [68]. Although this design choice reduces the flexibility of its smart contracts—for instance, proxy contracts and flashloans are largely limited—it enhances the network's credibility.

EOS, which supports smart contracts written in C++, also offers better scalability and flexibility. However, its reputation has suffered due to centralized governance and unmet promises. Unlike Solana, which addresses reentrancy at the design level, EOS remains vulnerable to reentrancy attacks. On May 14, 2021, a reentrancy exploit on EOS led to a $13 million loss [69]. Despite lacking a `fallback` function, EOS's message-passing model introduces similar vulnerabilities.

E. Applicability of Methodology to Other Blockchain Attacks

Our analysis relies primarily on the call traces of exploit transactions, which only include cross-contract operations such as `CALL`, `DELEGATECALL`, `STATICCALL`, and `CREATE`. Since reentrancy is inherently a cross-contract phenomenon, call traces are sufficient for detecting these attacks and locate the vulnerabilities. However, this approach does not generalize to many other vulnerability classes. For example, issues like integer overflow/underflow, timestamp dependency, and insufficient input validation typically occur within the internal logic of a single contract—below the granularity of the call trace. Although tools like Tenderly [34] support opcode-level inspection, such low-level traces are impractical to analyze at scale. Source code access becomes essential in these cases, but is not always available. In contrast, while Solidity source code enhances our analysis, it is not required; we successfully analyzed five incidents in our dataset using only call traces.

Furthermore, reentrancy is defined by clear signatures. By comparison, vulnerabilities like access control flaws or general logic bugs—top categories in OWASP's 2025 list [3]—are more abstract and context-dependent, making them harder to detect and categorize without deep semantic understanding and source code access. Attacks unrelated to contract logic, such as phishing, fall outside our methodology, as their root causes lie off-chain and require a different investigative approach.

F. Root Cause Analysis on Victim Contracts

Although this study adopts an attacker-centric view, analyzing vulnerabilities from the victim's perspective is equally important for prevention. During transaction analysis (Section IV-C), we identify root causes when source code is available, and we find that exploitable reentrancy vulnerabilities typically stem from a combination of two factors: 1) flawed logic that updates states *after* external calls (violating the CEI pattern), and 2) failure to account for malicious execution during seemingly benign transfer actions or underestimation of malicious intent in explicit external calls. Both conditions are equally necessary for successful exploitation. Although best practices advocate addressing both, developers often prioritize the second, as the increasing demand for flexibility often conflicts with the strict constraints of the CEI pattern. This tradeoff motivated our investigation of reentrancy entry points.

As for complete root cause analysis, it requires source code, commit histories, and audit records—resources often unavailable [70]. Moreover, as discussed in Section VI-F, vulnerabilities may originate in upstream dependencies rather than the victim contract itself. Tracing the origin of forked code and locating where the vulnerability is introduced is challenging without a systematic method for tracking project lineage. The identification of origins in Section VI-F relied on the researchers' prior knowledge and is likely incomplete.

G. Threats to Validity

An external threat is potential dataset bias, as our reentrancy attack data was sourced from publicly available repositories, which favor high-profile cases. Smaller-loss attacks might receive less attention and thus be underrepresented. We mitigate this by compiling data from multiple sources [4]–[6]. This challenge is not unique to our study; for instance, BlockWatchdog [14] also excludes attacks with losses below $100,000.

An internal threat is potential analysis bias, as both coauthors analyzed attack transactions using the same tools (BlockSec Phalcon Explorer [5] and Tenderly Explorer [34]) and reference materials (incident reports, victim post-mortems, and X/Twitter posts). Reliance on similar resources could introduce bias. To address this, we employed diverse sources and cross-validated findings (see Section IV-E).

VIII. Conclusion

This paper presents a comprehensive analysis of 73 real-world reentrancy attacks on EVM-compatible blockchains from 2016 to 2024. Using both qualitative and quantitative methods, we offered a multi-dimensional view of the evolving nature of these attacks, including shifts in vulnerability characteristics and attacker strategies. Our study goes beyond conventional classifications, highlighting reentrancy's dual role as both an exploit vector and a legitimate design pattern. It also challenges outdated assumptions and illustrates how attackers adapt to bypass established defenses. Our empirical evidence reveals critical trends: persistent attack frequency, increasingly complex scopes, evolving entry points beyond the `fallback` function, and rapid exploitation timelines. We further demonstrate that reentrancy is often combined with other vulnerabilities to magnify its impact.

Acknowledgment

We thank the white-hat researcher `pcaversaccio`, whose GitHub repository documenting reentrancy attacks served as the foundation for this study. This work was supported by the Natural Sciences and Engineering Research Council of Canada (NSERC), and the UBC Blockchain Pathway fellowship.

References

[1] V. Buterin, "Ethereum: A next-generation smart contract and decentralized application platform," Dec 2014. [Online]. Available: https://web.archive.org/web/20241212083005/https://ethereum.org/content/whitepaper/whitepaper-pdf/Ethereum_Whitepaper_-_Buterin_2014.pdf

[2] OWASP, "Owasp smart contract top 10 (2023)," Jun 2024. [Online]. Available: https://web.archive.org/web/20241130202135/https://owasp.org/www-project-smart-contract-top-10/

[3] ——, "Owasp smart contract top 10 (2025)," Jan 2025. [Online]. Available: https://web.archive.org/web/20250401011341/https://owasp.org/www-project-smart-contract-top-10/

[4] pcaversaccio, "Github - pcaversaccio/reentrancy-attacks: A chronological and (hopefully) complete list of reentrancy attacks to date." 2022. [Online]. Available: https://web.archive.org/web/20250105115624/https://github.com/pcaversaccio/reentrancy-attacks

[5] BlockSec, "Security incidents — phalcon explorer," 2024. [Online]. Available: https://app.blocksec.com/explorer/security-incidents

[6] SlowMist, "Slowmist hacked - slowmist zone," 2024. [Online]. Available: https://web.archive.org/web/20250122072857/https://hacked.slowmist.io/?c=Blockchain

[7] E. Weippl, S. Katzenbeisser, C. Kruegel, A. Myers, S. Halevi, L. Luu, D.-H. Chu, H. Olickel, P. Saxena, and A. Hobor, "Making smart contracts smarter," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, ser. Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Oct 2016, pp. 254–269. [Online]. Available: https://dl.acm.org/doi/abs/10.1145/2976749.2978309

[8] A. Ghaleb and K. Pattabiraman, "How effective are smart contract analysis tools? evaluating smart contract static analysis tools using bug injection," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2020. New York, NY, USA: Association for Computing Machinery, 2020, p. 415–427. [Online]. Available: https://doi.org/10.1145/3395363.3397385

[9] J. Feist, G. Greico, and A. Groce, "Slither: A Static Analysis Framework for Smart Contracts," in *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain*, ser. 2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain, vol. 00, Sep 2019, pp. 8–15. [Online]. Available: https://ieeexplore.ieee.org/document/8823898

[10] M. Rodler, W. Li, G. O. Karame, and L. Davi, "Sereum: Protecting Existing Smart Contracts Against Re-Entrancy Attacks," *arXiv*, Dec 2018. [Online]. Available: https://arxiv.org/abs/1812.05934

[11] "Security considerations - solidity 0.8.28 documentation," May 2023. [Online]. Available: https://web.archive.org/web/20250424033851/https://docs.soliditylang.org/en/v0.8.28/security-considerations.html#reentrancy

[12] S. Vishwas, "Solidity programming: The checks-effects-interactions pattern," Mar 2023. [Online]. Available: https://web.archive.org/web/20250424034132/https://samvishwas.medium.com/solidity-programming-the-checks-effects-interactions-pattern-bac0e5a5f659

[13] A. Roychoudhury, A. Paiva, R. Abreu, M. Storey, S. Yang, J. Chen, M. Huang, Z. Zheng, and Y. Huang, "Uncover the premeditated attacks: Detecting exploitable reentrancy vulnerabilities by identifying attacker contracts," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, Apr 2024, pp. 1573–1584. [Online]. Available: https://dl.acm.org/doi/abs/10.1145/3597503.3639153

[14] Z. Zheng, N. Zhang, J. Su, Z. Zhong, M. Ye, and J. Chen, "Turn the rudder: A beacon of reentrancy detection for smart contracts on ethereum," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, Jul 2023, pp. 295–306. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10172623

[15] CoinGecko, "Top blockchains ranked by total value locked (tvl) — coingecko," 2024. [Online]. Available: https://web.archive.org/web/20241207215500/https://www.coingecko.com/en/chains

[16] Etherscan, "Ethereum (eth) blockchain explorer," 2019. [Online]. Available: https://etherscan.io/

[17] F. Vogelsteller and V. Buterin, "Eip 20: Erc-20 token standard," Nov 2015. [Online]. Available: https://web.archive.org/web/20241210055610/https://eips.ethereum.org/EIPS/eip-20

[18] J. Dafflon, J. Baylina, and T. Shababi, "Eip 777: Erc777 token standard," Nov 2017. [Online]. Available: https://web.archive.org/web/20241205154726/https://eips.ethereum.org/EIPS/eip-777

[19] W. Entriken, D. Shirley, J. Evans, and N. Sachs, "Eip 721: Erc-721 non-fungible token standard," Jan 2018. [Online]. Available: https://web.archive.org/web/20241210131044/https://eips.ethereum.org/EIPS/eip-721

[20] W. Radomski, A. Cooke, P. Castonguay, J. Therien, E. Binet, and R. Sandford, "Eip 1155: Erc-1155 multi token standard," Jun 2018. [Online]. Available: https://web.archive.org/web/20241223163544/https://eips.ethereum.org/EIPS/eip-1155

[21] A. Minaev, "The dao hack explained — all the details of infamous attack," Jun 2023. [Online]. Available: https://web.archive.org/web/20241210091318/https://cryptodose.net/learn/the-dao-hack/

[22] T. Jiao, Z. Xu, M. Qi, S. Wen, Y. Xiang, and G. Nan, "A survey of ethereum smart contract security: Attacks and detection," *Distributed Ledger Technologies: Research and Practice*, vol. 3, no. 3, pp. 1–28, Sep 2024. [Online]. Available: https://dl.acm.org/doi/full/10.1145/3643895

[23] M. Mossberg, F. Manzano, E. Hennenfent, A. Groce, G. Grieco, J. Feist, T. Brunson, and A. Dinaburg, "Manticore: A user-friendly symbolic execution framework for binaries and smart contracts," in *2019 34th IEEE/ACM International Conference on Automated Software Engineering*, ser. 2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE), vol. 00. IEEE, Jan 2020, pp. 1186–1189. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/8952204

[24] S. Kalra, S. Goel, M. Dhawan, and S. Sharma, "Zeus: Analyzing safety of smart contracts," in *Ndss*, 2018, pp. 1–12. [Online]. Available: https://sukritkalra.github.io/data/papers/zeus.pdf

[25] D. Lie, M. Mannan, M. Backes, X. Wang, P. Tsankov, A. Dan, D. Drachsler-Cohen, A. Gervais, F. Bünzli, and M. Vechev, "Securify: Practical security analysis of smart contracts," in *2018 ACM SIGSAC Conference on Computer and Communications Security*, ser. Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, Oct 2018, pp. 67–82. [Online]. Available: https://dl.acm.org/doi/abs/10.1145/3243734.3243780

[26] L. Brent, A. Jurisevic, M. Kong, E. Liu, F. Gauthier, V. Gramoli, R. Holz, and B. Scholz, "Vandal: A scalable security analysis framework for smart contracts," *arXiv*, Sep 2018. [Online]. Available: https://arxiv.org/abs/1809.03981

[27] S. Palladino, F. Giordano, and H. Croubois, "Eip-1967: Standard proxy storage slots," Apr 2019. [Online]. Available: https://web.archive.org/web/20241223163545/https://eips.ethereum.org/EIPS/eip-1967

[28] M. Chaudron, I. Crnkovic, M. Chechik, M. Harman, C. Liu, H. Liu, Z. Cao, Z. Chen, B. Chen, and B. Roscoe, "Reguard: Finding reentrancy bugs in smart contracts," in *40th International Conference on Software Engineering: Companion Proceeedings*, ser. Proceedings of the 40th International Conference on Software Engineering: Companion Proceeedings, May 2018, pp. 65–68. [Online]. Available: https://dl.acm.org/doi/abs/10.1145/3183440.3183495

[29] M. Huchard, C. Kästner, G. Fraser, B. Jiang, Y. Liu, and W. K. Chan, "Contractfuzzer: Fuzzing smart contracts for vulnerability detection," in *33rd ACM/IEEE International Conference on Automated Software Engineering*, ser. Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering, vol. 00, Sep 2018, pp. 259–269. [Online]. Available: https://dl.acm.org/doi/abs/10.1145/3238147.3238177

[30] X. Zhao, H. Qu, J. Xu, X. Li, W. Lv, and G.-G. Wang, "A systematic review of fuzzing," *Soft Computing*, vol. 28, no. 6, pp. 5493–5522, 2024. [Online]. Available: https://link.springer.com/article/10.1007/s00500-023-09306-2

[31] J. Zhang, Z. Zheng, Y. Nan, M. Ye, K. Ning, Y. Zhang, and W. Zhang, "Smartreco: Detecting read-only reentrancy via fine-grained cross-dapp analysis," *arXiv*, Sep 2024. [Online]. Available: https://arxiv.org/abs/2409.18468

[32] J. Grundy, C. L. Goues, D. Lo, Y. Xue, M. Ma, Y. Lin, Y. Sui, J. Ye, and T. Peng, "Cross-contract static analysis for detecting practical reentrancy vulnerabilities in smart contracts," in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, Oct 2020, pp. 1029–1040. [Online]. Available: https://dl.acm.org/doi/abs/10.1145/3324884.3416553

[33] Y. Liu, X. Rui, and K. Pattabiraman, "Dataset and reproducibility for reentrancy redux: The evolution of real-world reentrancy attacks on blockchains," Mar 2025. [Online]. Available: https://doi.org/10.5281/zenodo.15112729

[34] Tenderly, "Tenderly explorer," 2024. [Online]. Available: https://dashboard.tenderly.co/explorer

[35] Y. Tang, Z. Li, and Y. Bai, "Rethinking of reentrancy on the ethereum," in *2021 IEEE Intl Conf on Dependable, Autonomic and Secure Computing, Intl Conf on Pervasive Intelligence and Computing, Intl Conf on Cloud and Big Data Computing, Intl Conf on Cyber Science and Technology Congress (DASC/PiCom/CBDCom/CyberSciTech)*, vol. 00, Mar 2022, pp. 68–75. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9730357

[36] Solidus Labs, *The 2022 Rug Pull Report*. Solidus Labs, Dec 2022. [Online]. Available: https://web.archive.org/web/20241226164934/https://www.soliduslabs.com/reports/rug-pull-report

[37] P. Kacherginsky, "Blockthreat - week 17, 2022," May 2022. [Online]. Available: https://web.archive.org/web/20250122113904/https://newsletter.blockthreat.io/p/blockthreat-week-17-2022

[38] Neptune Mutual, "Analysis of the barley finance exploit," Mar 2024. [Online]. Available: https://web.archive.org/web/20250424032214/https://medium.com/neptune-mutual/analysis-of-the-barley-finance-exploit-d2df61b98c80

[39] L. Risk, "Curve pool reentrancy exploit postmortem july 30th, 2023 - hackmd," Aug 2023. [Online]. Available: https://web.archive.org/web/20241208034705/https://hackmd.io/@LlamaRisk/BJzSKHNjn

[40] Vyperlang, "Vyper nonreentrancy lock vulnerability technical post-mortem report - hackmd," Aug 2023. [Online]. Available: https://web.archive.org/web/20241213102337/https://hackmd.io/@vyperlang/HJUgNMhs2

[41] Rarma, "Terra ibc hooks exploit," Aug 2024. [Online]. Available: https://web.archive.org/web/20241208081243/https://medium.com/@Rarma_/terra-ibc-hooks-exploit-21cca87d70a4

[42] H. Eng, "Not just a bank run: New evidence shows iron finance crashed due to code exploit," Jun 2021. [Online]. Available: https://web.archive.org/web/20241209040226/https://thedefiant.io/news/research-and-opinion/not-just-a-bank-run-new-evidence-shows-iron-finance-crashed-due-to-code-exploit

[43] ChainList, "Chainlist," Oct 2024. [Online]. Available: https://web.archive.org/web/20241207211327/https://chainlist.org/

[44] pcaversaccio, "Smart contract deployment statistics," Apr 2023. [Online]. Available: https://dune.com/pcaversaccio/smart-contract-deployment-statistics

[45] Shashank, "Jay token exploit - reentrancy attack," Jan 2023. [Online]. Available: https://web.archive.org/web/20241205043544/https://blog.solidityscan.com/jay-token-exploit-reentrancy-attack-d7a4923b6333

[46] SlowMist, "Slowmist: Details of lendf.me reentrancy attack," Apr 2020. [Online]. Available: https://web.archive.org/web/20241207005214/https://slowmist.medium.com/slowmist-details-of-lendf-me-reentrancy-attack-3e168ab5f2b1

[47] Alan, "Exploitedtoadz, a technical deepdive," Oct 2021. [Online]. Available: https://web.archive.org/web/20241207005527/https://medium.com/@ItsCuzzo/exploitedtoadz-a-technical-deepdive-9ceabf46d0ce

[48] BlockSec, "When "safemint" becomes unsafe: Lessons from the hypebears security incident," Feb 2022. [Online]. Available: https://web.archive.org/web/20241207010234/https://blocksecteam.medium.com/when-safemint-becomes-unsafe-lessons-from-the-hypebears-security-incident-2965209bda2a

[49] SlowMist, "Revest finance incident analysis," Mar 2022. [Online]. Available: https://web.archive.org/web/20241207005731/https://slowmist.medium.com/revest-finance-incident-analysis-6fcd9b6be207

[50] C. Girdhar, "Metaverse project paraluni hacked: Reportedly $1.7m lost," Mar 2022. [Online]. Available: https://web.archive.org/web/20241207010953/https://coincodecap.com/paraluni-hacked-reportedly-1-7m-lost

[51] Rekt, "Grim finance - rekt," Dec 2021. [Online]. Available: https://web.archive.org/web/20241207005748/https://rekt.news/grim-finance-rekt

[52] Amber Group, "Preventing re-entrancy attacks - lessons from history," Sep 2021. [Online]. Available: https://web.archive.org/web/20250105114210/https://medium.com/amber-group/preventing-re-entrancy-attacks-lessons-from-history-c2d96480fac3

[53] Abracadabra Money, "Cauldron v4 post mortem," 2023. [Online]. Available: https://web.archive.org/web/20241207005250/https://mirror.xyz/0x5744b051845B62D6f5B6Db095cc428bCbBBAc6F9/47LK6nUpMrVsYzfCYBTyZsc_7t5Sh5onxO8sSEotNMY

[54] C. Shou, Y. Ke, Y. Yang, Q. Su, O. Dadosh, A. Eli, D. Benchimol, D. Lu, D. Tong, D. Chen *et al.*, "Backrunner: Mitigating smart contract attacks in the real world," *arXiv*, 2024. [Online]. Available: https://arxiv.org/abs/2409.06213

[55] D. Verheijke and H. Rocha, "An exploratory study on solidity guards and ether exchange constructs," in *Proceedings of the 5th International Workshop on Emerging Trends in Software Engineering for Blockchain*, 2022, pp. 1–8. [Online]. Available: https://dl.acm.org/doi/abs/10.1145/3528226.3528372

[56] A. Roan, "How spankchain got hacked," Mar 2020. [Online]. Available: https://web.archive.org/web/20241207005350/https://medium.com/swlh/how-spankchain-got-hacked-af65b933393c

[57] "Smart contract security field guide - reentrancy," Feb 2025. [Online]. Available: https://web.archive.org/web/20241204130326/https://scsfg.io/hackers/reentrancy/

[58] "The ultimate guide to reentrancy," May 2023. [Online]. Available: https://web.archive.org/web/20241224174646/https://medium.com/immunefi/the-ultimate-guide-to-reentrancy-19526f105ac

[59] R. Rahimian and J. Clark, "Tokenhook: Secure erc-20 smart contract," *arXiv preprint arXiv:2107.02997*, 2021. [Online]. Available: https://arxiv.org/abs/2107.02997

[60] K. Nelaturu, A. Mavridou, E. Stachtiari, A. Veneris, and A. Laszka, "Correct-by-design interacting smart contracts and a systematic approach for verifying erc20 and erc721 contracts with verisolid," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 4, pp. 3110–3127, 2022. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9864250

[61] T. Be'ery, "imbtc & erc777: Defi hack explained," Apr 2020. [Online]. Available: https://web.archive.org/web/20250123171447/https://zengo.com/imbtc-defi-hack-explained/

[62] "Why doesn't solidity disable reentrant behavior at the compiler level?" August 2023. [Online]. Available: https://www.reddit.com/r/ethdev/comments/15hmwze/why_doesnt_solidity_disable_reentrant_behavior_at/

[63] ImmuneBytes, "Shield your smart contracts with a robust reentrancy guard," Sep 2022. [Online]. Available: https://web.archive.org/web/20250424034813/https://immunebytes.com/blog/shield-your-smart-contracts-with-a-robust-reentrancy-guard/

[64] QuillAudits, "Decoding $220k read-only reentrancy exploit," Oct 2022. [Online]. Available: https://web.archive.org/web/20241207005218/https://quillaudits.medium.com/decoding-220k-read-only-reentrancy-exploit-quillaudits-30871d728ad5

[65] Shashank, "Jarvis polygon pool hack analysis — read only re-entrancy," Jan 2023. [Online]. Available: https://blog.solidityscan.com/jarvis-polygon-pool-hack-analysis-read-only-re-entrancy-af0607e4585a

[66] QuillAudits, "Decoding sentiment protocol's $1 million exploit," Apr 2023. [Online]. Available: https://web.archive.org/web/20241207005752/https://quillaudits.medium.com/decoding-sentiment-protocols-1-million-exploit-quillaudits-f36bee77d376

[67] P. Sharma, "Solana vs. ethereum: the battle for dominance," Nov 2024. [Online]. Available: https://web.archive.org/web/20250424035103/https://www.analyticsinsight.net/ethereum/solana-vs-ethereum-the-battle-for-dominance

[68] 0xGuard, "Solana. top vulnerabilities," 2022. [Online]. Available: https://web.archive.org/web/20250424035334/https://defisec.info/solana_top_vulnerabilities

[69] EOS Costa Rica, "Analysis of the vulnerability found in the vaults.sx smart contract," Jun 2021. [Online]. Available: https://web.archive.org/web/20241209034252/https://eoscostarica.medium.com/analysis-of-the-vulnerability-found-in-the-vaults-sx-smart-contract-445c8c968b5f

[70] Y. Wang, X. Chen, Y. Huang, H.-N. Zhu, J. Bian, and Z. Zheng, "An empirical study on real bug fixes from solidity smart contract projects," *Journal of Systems and Software*, vol. 204, p. 111787, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0164121223001826